



Bastien Golomer :: Trainee :: Paul Scherrer Institute

Differentiable influence matrix calculation using automatic differentiation

CPT Experimental Project Meeting, 20.02.2023

1. FlonA and the problem of differentiability

- Why differentiability ?
- Quick reminder : FlonA's dose calculation.
- Principle of automatic differentiation

2. Juliana, a differentiable dose calculation

- Setup : libraries, Fiona and assumptions
- WED calculation
- Dij elements calculation

3. Results

WED for 1 patient

Dose distribution for 3 patients

4. Outlook

- Improvements
- How to check the gradients

Acronyms

- OP = Optimization Point
- WED = Water Equivalent Depth
- MCS = Multiple Coulomb Scattering
- ML = Machine Learning

1. FIonA and the problem of differentiability

Why differentiability?

- **Goal** : Optimize spot positions, energies and field angles

- **Approach** :

In a simplified way:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} L(\mathbf{d}(\mathbf{w}, \alpha))$$

L := loss function.

$\mathbf{d}(\mathbf{w}, \alpha)$:= dose vector (each optimization point)

$\mathbf{w}$:= spot positions, energies and angles : **variables**

α := parameters for dose calculation : **constants**

$\mathbf{w}^*$:= optimal solution

Gradient Descent techniques to find $\mathbf{w}^*$.

→ $\nabla_{\mathbf{w}} L(\mathbf{d}(\mathbf{w}, \alpha))$

→ How to get?

Chain rule ! : $\nabla_{\mathbf{w}} L(\mathbf{d}(\mathbf{w}, \alpha)) = \nabla_{\mathbf{d}} L(\mathbf{d}) \cdot \mathbf{J}_{\mathbf{d}}(\mathbf{w}, \alpha)$

$\nabla_{\mathbf{d}} L(\mathbf{d})$ easy to get.

$\mathbf{J}_{\mathbf{d}}(\mathbf{w}, \alpha) :=$ Jacobian matrix

This work !

Why change ?

Current pipeline not differentiable:

Look ups & Indices.

Solution

- rewrite kernels in differentiable way!

- Use Automatic differentiation,
yields $\mathbf{J}_{\mathbf{d}}(\mathbf{w}, \alpha)$!

FlonA's Dose calculation

Inputs : Plan (fields), CT, LUTs

Hu to Sp conversion ,
Bragg Peaks, MCS
scurves, Initial phase
space spreads, nozzle/
absorber phase spaces

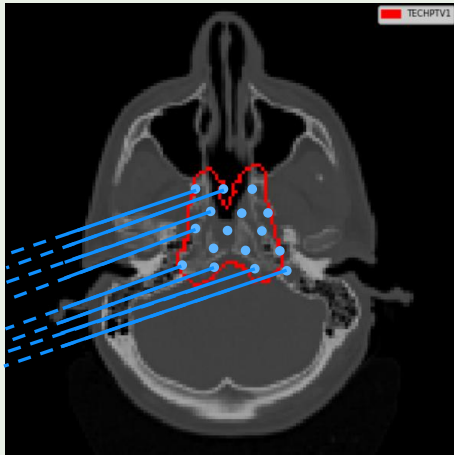
Optimizers :

OPs, Spot positions,
Energies & weights $\{w_j\}$

Water Equivalent Depth L

- Optimization point
- Source - OP direction

Fig.1.1 : Representation of some OPs and source direction for Patient 0



For each OP i :

- Compute first intersection with CT
- For each voxel crossed k :

1. Intersections
2. Distance in voxel δ_k
3. Voxel density ρ_k (look up)
4. Multiply and add

$$L_i = \sum_{k:=\text{voxels}} \delta_k \rho_k$$

Influence matrix element D_{ij}

$$D_{ij} = N(L_i, \sigma_{MCS}^2 + \sigma_{IPSt}^2 + \sigma_{IPSu}^2) \cdot D_{\infty}(L_i, E_j)$$

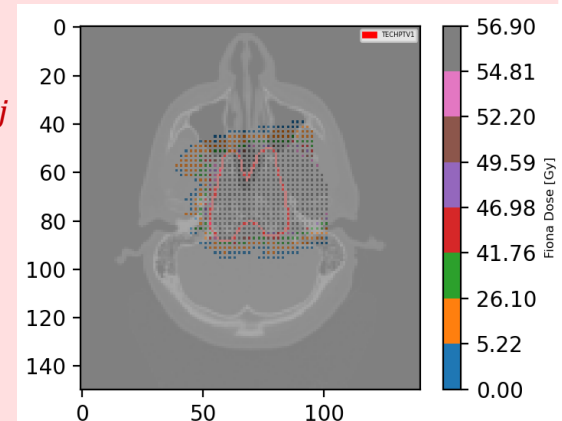
For each OP i and spot j :

- Initial Phase Space spreads $\sigma_{IPSt}, \sigma_{IPSt}$ (look ups)
- If spot j affects point i :
 - σ_{MCS} (look up)
 - Dose (look up)

Dose d_i

$$d_i = \sum_{j:=\text{spots}} D_{ij} w_j$$

Fig1.2 : Dose distribution for Patient 0 computed with FlonA



Automatic Differentiation Principle

Core concept

- Any function is a **composition of basic function** and operations: +, -, /, ·, exp, ln, cos, sin ...
- Each composition $\text{:= primal } v \rightarrow$ **Graph representation**

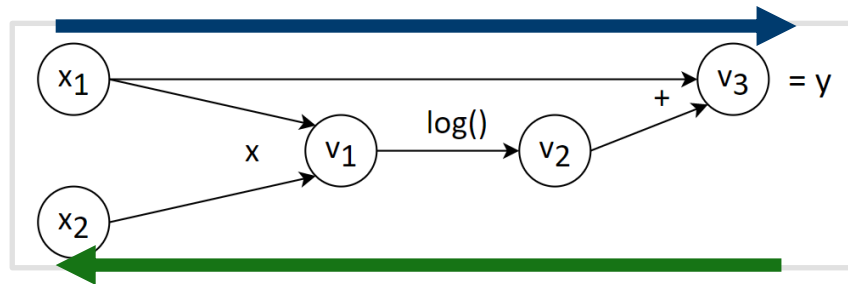


Fig1.3: Graph of primals for function $f(x_1, x_2) = \log(x_1 x_2) + x_1 = y$

- **First pass** : stores primal values. **Compose primals to get result**
- **Second pass** : **Chain rule** over primals. Stores values on the fly.

2 Approaches to the Chain Rule :

Forward mode vs **Backward mode**, consider either:

- Derivatives $\text{:= sensitivity of a considered primal with respect to an input.}$
- Adjointes $\text{:= sensitivity of an output with respect to a considered primal.}$

Practically :

Forward : 1 pass per input (left-right) => for **few inputs, large outputs**

Backward : 1 pass per output (right-left) for **large inputs, few outputs. Adapted to ML.**

2. Juliana, a differentiable dose calculation

- **Julia 1.8.0**

Suited for numerical analysis and computational sciences.

High performance

- **Enzyme.jl [1][2] :**

- Allows **differentiable GPU kernels**

- Performs **Activity analysis**

Labels “**active**” the operations that impact derivative computation

=> For every **active operation**, store **intermediate values**

- **KernelAbstractions.jl [3]**

-> Independence from GPU technology (compatible with CUDA & ROCm)

Juliana's Dose calculation

Inputs : Plan (fields), CT, LUTs

FlonA : Optimization Points, Spot positions, energies & weights $\{w_j\}$, WED

Assumptions : 1 field, HFS position, no preabsorber, CT origin at (0,0,0).

1) Water Equivalent Depth L

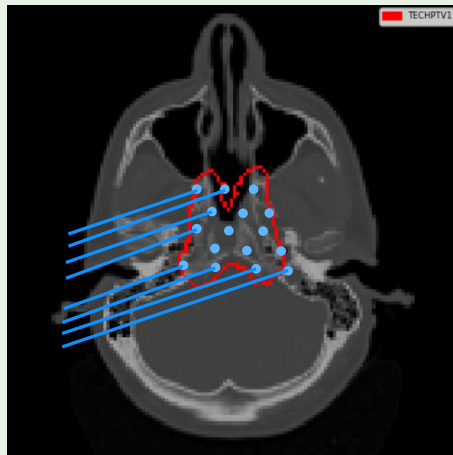


Fig.2.1 : Representation of some OPs and source direction for Patient 0.

- **Differentiability** = Trilinear interpolation of density matrix. Continuous representation.
- Select fixed step size δ to travel trajectory

Kernel

For each OP i :

1. Place source at infinity
2. For each iteration along trajectory:
 - Trilinear interpolation of density ρ_k
 - Multiply by step size
 - Accumulate

$$L_i = \sum_{k:=\text{steps}} \delta \cdot \rho_k$$

Active values stored at each iteration !

2) Influence matrix elements $\{D_{ij}\}$

$$\text{Recall : } D_{ij} = N(L, \sigma_{MCS}^2 + \sigma_{IPSt}^2 + \sigma_{IPSu}^2) \cdot D_{\infty}(L_i, E_j)$$

Non Differentiability : Look Up Tables and indices : **discrete**

-> push idea of **linear interpolation**

Problem : Look Up Tables have **irregular energy increment**

- Gradient w.r.t spots energy requires **continuum of energy**,
- Irregular increment in energy => we **need lookup of tabulated energies** to find index of correct values for interpolation, to be **avoided**
- Need **preprocessing** before kernel

Preprocessing

1) Initial Phase Space spreads : $\sigma_{IPSt}, \sigma_{IPSu}$

- Linear interpolation in energy -> regular increment
- Pass interpolated coefficients to kernel for interpolation inside.

2) MCS curves ?

3) Depth-Dose curves ?

Multiple Coulomb Scattering spread

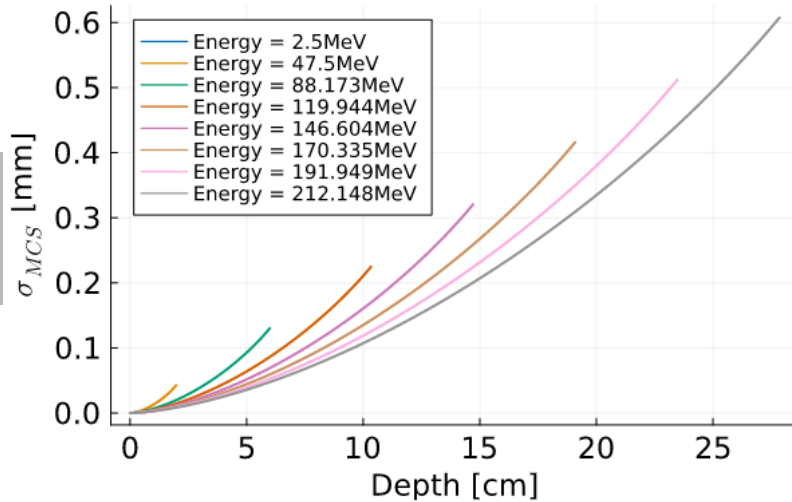


Fig. 2.2 Tabulated beam spreads due to Multiple Coulomb Scattering (MCS) at CPT, selected energies.

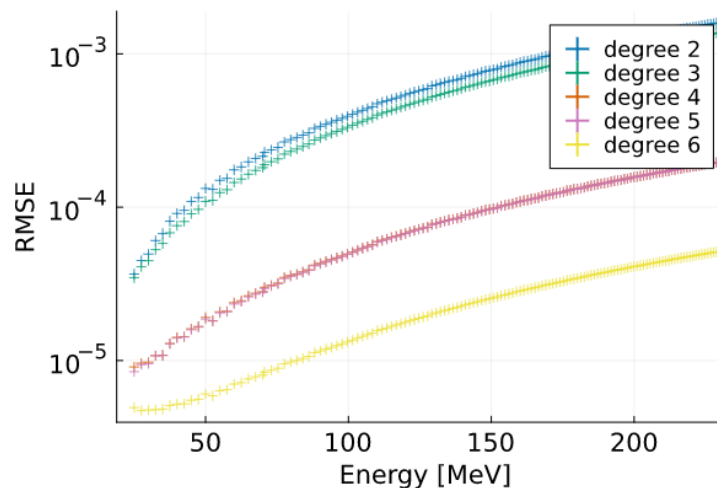


Fig. 2.3 Root Mean Square error between all MCS curves and polynomial fits of increasing degree.

Approach :

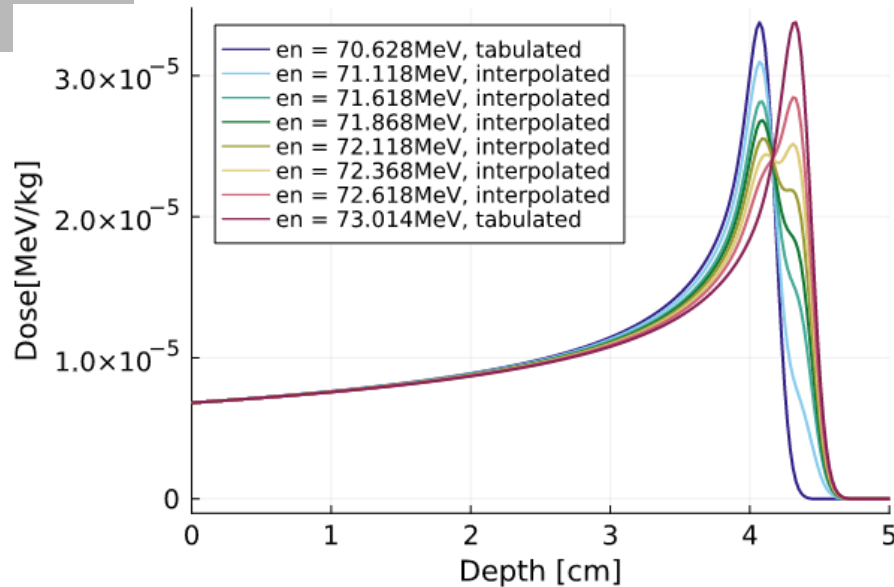
1. Curves have a simple shape -> **Polynomial fit**

$$\sigma_{MCS}(s, E) = \sum_{i=0}^{\infty} C_i(E) \cdot s^i$$

2. Check accuracy of fits with Root Mean Square Error.
3. Choose Degree 4 a good tradeoff between accuracy and number of inputs.
4. Interpolate $C_i(E)$ in energy (continuum)

Depth dose curves, which interpolation ?

Problem: Peaks **too sharp**, increment **too large**, bilinear interpolation preforms poorly... : **Saddles**



Short term solution

- Dose curves are interpolated along the **depth only**
- No need for preprocessing
- **Lose gradient w.r.t energy**

Long term solution

- Parametrization of the peaks
- Explore alternative interpolations (inverse distance)

Fig. 2.5 Behavior of the interpolated curves between two tabulated curves. The tabulated curves are from CPT. Bi-linear interpolation is performed over depth and energy.

2) Final Procedure

Preparation

- **Initial Phase Space** σ_{IPSt} , σ_{IPSu} , Linear interpolation in energy.
- **MCS spread** : 4th order fit, Coefficients interpolated w.r.t. energy
- **Depth dose curves** : no preprocessing

Computation

- **Initial phase Space** : Linear interpolation w.r.t energy
- Check if spot reaches optimization point, then :
 - **MCS** spread from fitted polynomial coefficients
 - **Dose** from linear interpolation w.r.t depth

3. Results

Results - Water Equivalent Depth

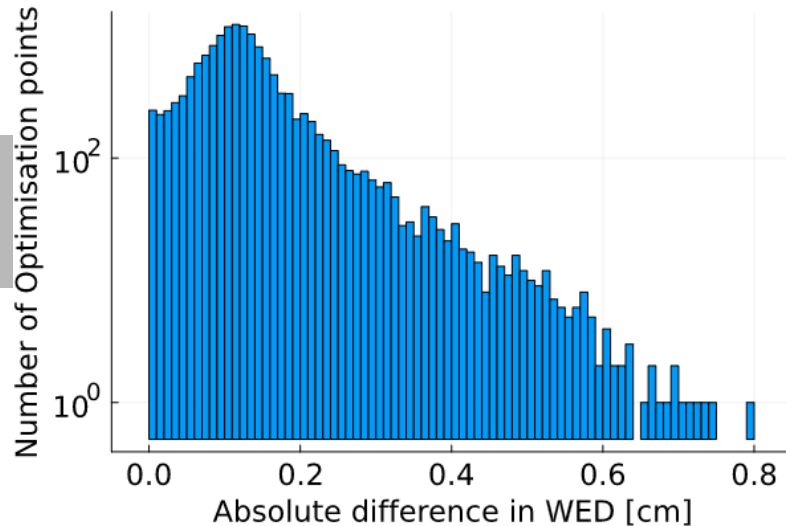


Fig 3.1 Histogram of difference in WED between Fiona and Juliana for each OP. Patient 0.

Statistics	WED calculation[μ s]	Gradient calculation [s]
Mean	177.5	3.95
std	29.5	$13 \cdot 10^{-3}$
Median	171.3	3.98
min	155.1	3.95
max	497.7	4.01

Table 3.1 Benchmark statistics of WED and calculations. Patient 0 is used.

Problems

- Discrepancies too high
- Gradient computation needs Batches: primals & adjoint overflow memory
- Batch approach too long for ML applications

Solution in progress (see Discussion):

- Accuracy : subvoxel method
- Speed : Write own differentiation rule

Results – Dose distribution Patient 0

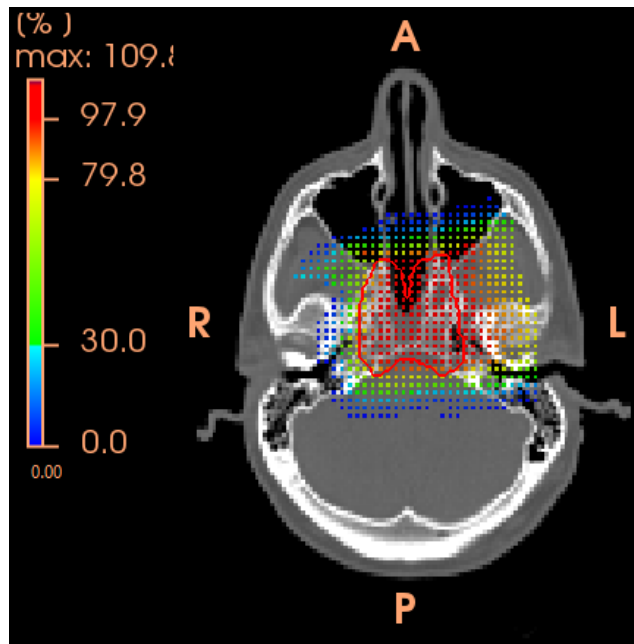


Fig. 3.2: Dose distribution in % of the prescribed dose computed by FlonA for Patient 0. CT slice 41.

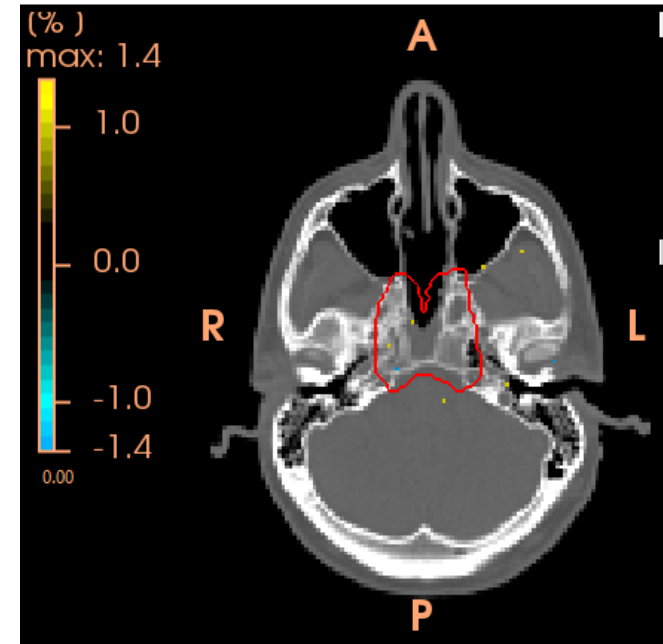


Fig. 3.3: Dose distribution difference between FlonA and Juliana in % for Patient 0. The reference is the dose from FlonA. CT slice 41.

Patient 0 - Plan

Gantry angle[degree]	75.0
Couch angle[degree]	15.0
Prescribed dose [Gy]	73.8
Nozzle Extraction	15.0
Target structure	TECHPTV1

Table 3.2:
Plan for
Patient 0.

Patient 0 - Statistics

Max pos. diff. [%]	1.355
Max neg. diff. [%]	-1.355
Mean diff. [%]	0.001
Voxel within $\pm 1\%$ [%]	98.724
Voxel within $\pm 3\%$ [%]	100

Table 3.3:
Difference
statistics for
Patient 0.

Results – Dose distribution Patient 1

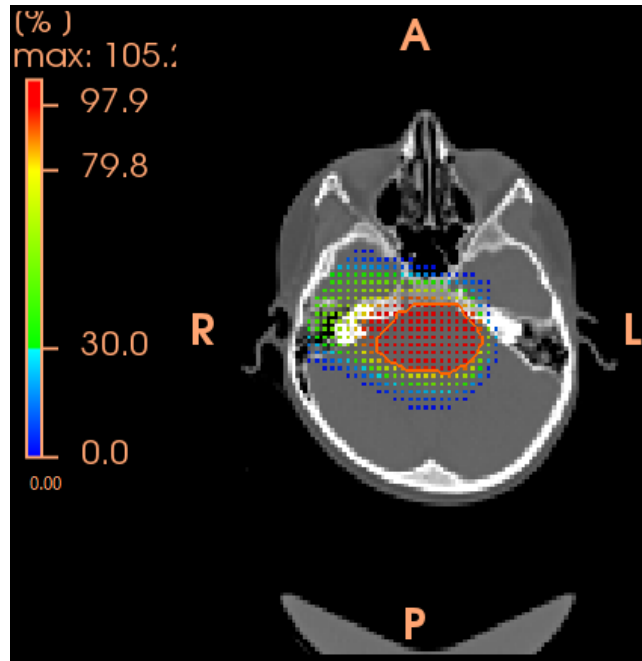


Fig. 3.4: Dose distribution in % of the prescribed dose computed by FlonA for Patient 1. CT slice 39

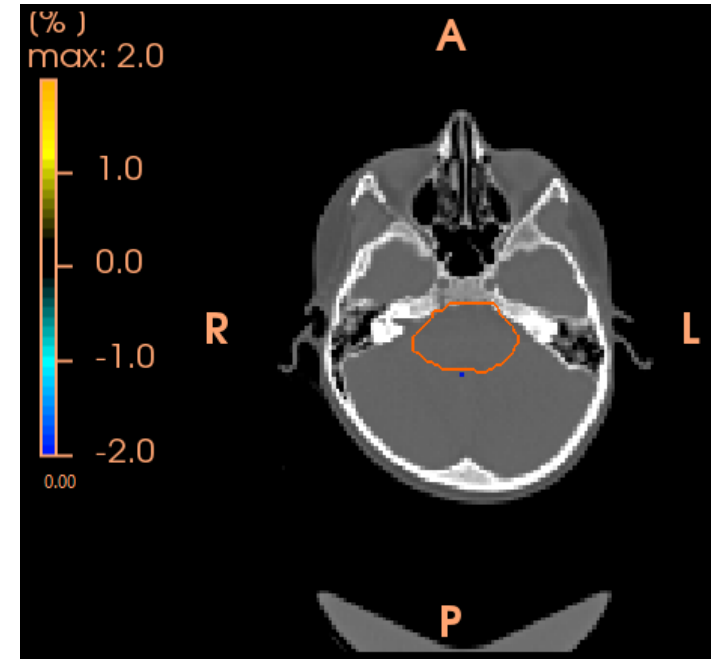


Fig. 3.5: Dose distribution difference between FlonA and Juliana in % for Patient 1. The reference is the dose from FlonA. CT slice 39

Patient 1 - Plan

Gantry angle[degree]	70.0
Couch angle[degree]	165.0
Prescribed dose [Gy]	50.4
Nozzle Extraction [cm]	1.0
Target structure	PTV

Table 3.4:
Plan for
Patient 1

Patient 1 – Statistics

Max pos. diff. [%]	1.984
Max neg. diff. [%]	-1.984
Mean diff. [%]	0.001
Voxel within $\pm 1\%$ [%]	99.198
Voxel within $\pm 3\%$ [%]	100

Table 3.5:
Difference
statistics for
Patient 1

Results – Dose distribution Patient 2

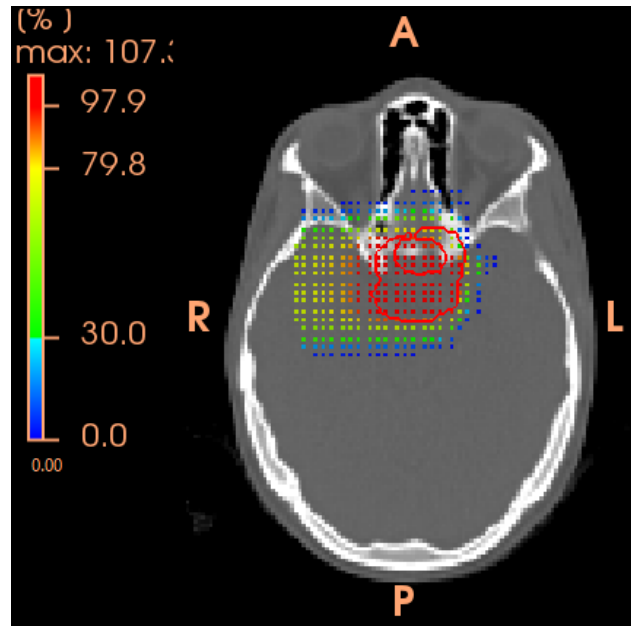


Fig. 3.6: Dose distribution in % of the prescribed dose computed by FlonA for Patient 2. CT slice 55.

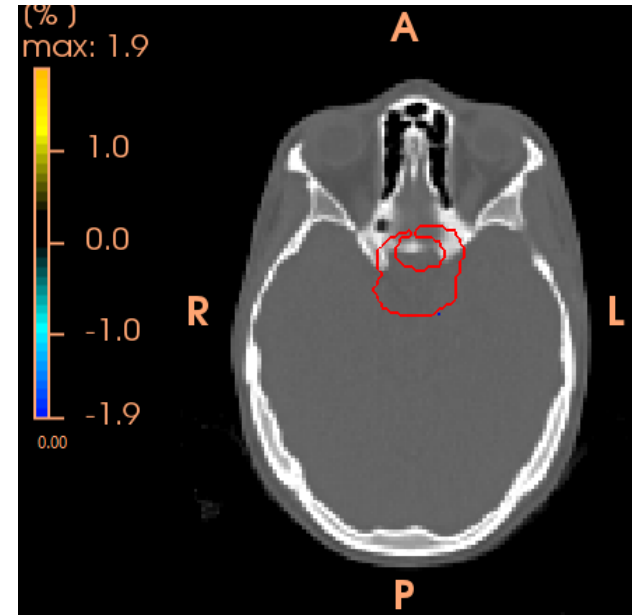


Fig. 3.7: Dose distribution difference between FlonA and Juliana in % for Patient 2. The reference is the dose from FlonA. CT slice 55.

Patient 2 - Plan

Gantry angle[degree]	95.0
Couch angle[degree]	180.0
Prescribed dose [Gy]	52.2
Nozzle Extraction [cm]	15.0
Target structure	TECHPTV1

Table 3.6:
Plan for
Patient 2

Patient 2 - Statistics

Max pos. diff. [%]	1.917
Max neg. diff. [%]	-1.915
Mean diff. [%]	0.001
Voxel within $\pm 1\%$ [%]	99.472
Voxel within $\pm 3\%$ [%]	100

Table 3.7:
Difference
statistics for
Patient 2

Benchmarks dose distribution calculation

Statistics	Dose calculation[μ s]	Gradient calculation [ms]
Mean	127.7	261.4
std	6.0	5
Median	126.9	263.3
min	114.5	252.8
max	155.4	286.2

Table 3.8 Benchmark statistics of WED and calculations. Patient 0 is used.

Gradient in ¼ seconds : to be tested with gradient descent

4. Discussion & Outlook

1) WED Accuracy : Subvoxel Method

- Introduce scaling factor γ (5 in example)
- Divide into γ^3 subvoxels
- Perform Trilinear interpolation over subvoxels at each step
- Tradeoff number of steps and accuracy

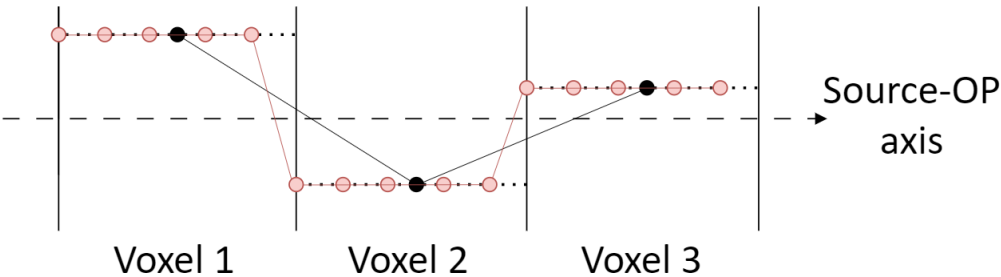


Fig 2. 1D simplification of the subvoxel method in the WED calculation. Here the scaling factor is set to 5. Dotted line is the CT number profile in each voxel.

2) WED Memory & Speed : Custom rule

- WED to Mathematical expression
- Differentiate manually
- Write a “Rule” using ChainRulesCore.jl

3) Depth-Dose curves

- Parametrization of the peaks
- Explore alternative interpolations (inverse distance)

1) Check on the gradient:

- Use Water Cuboid Phantom
- Place **1 spot outside**
- Compute gradient
- Gradient should lead to optimal placement in Phantom

2) Spot minimalism

- Start with **small number** of spots
- **Optimise** positions and energies
- **Increase number** until no improvement in dose repartition
- > Get **optimal number**, positions and energies ! (bottom up approach)

- FlonA's current dose calculation **not differentiable**
- Introduce **Automatic differentiation**
- **Differentiable** pipeline produced
- **Water Equivalent Depth**
 - Trilinear interpolation
 - Differentiation rule for speed
 - Subvoxel method for accuracy
- **Acceptable Dose calculation**
 - Polynomial fit of MCS curves
 - Depth linear interpolation of Dose curves, need more definition
 - Mean difference to FlonA = 0.001% for each test patient
 - Voxel with difference $\pm 1\%$ > 98.5 %
 - Benchmark statistics

- [1] Moses, W., & Churavy, V. (2020). Instead of Rewriting Foreign Code for Machine Learning, Automatically Synthesize Fast Gradients. In *Advances in Neural Information Processing Systems* (pp. 12472–12485). Curran Associates, Inc..
- [2] Moses, W., Churavy, V., Paehler, L., Hückelheim, J., Narayanan, S., Schanen, M., & Doerfert, J. (2021). Reverse-Mode Automatic Differentiation and Optimization of GPU Kernels via Enzyme. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Association for Computing Machinery.
- [3] Churavy, V. & Contributors, KernelAbstractions.jl,
<https://github.com/JuliaGPU/KernelAbstractions.jl>

Special thanks to

- Renato Bellotti
- Andreas Adelmann
- Tony Lomax
- CPT PhD and Master students
- AMAS PhD and Master students





Questions ?

Appendix : Example for intuition

$$f(x_1, x_2) = \log(x_1 x_2) + x_1 = y$$

$$\partial_{x_1} f(x_1, x_2) = \frac{1}{x_1} + 1$$

$$\partial_{x_1} f \Big|_{x_1=5, x_2=2} = ? = \frac{1}{5} + 1$$

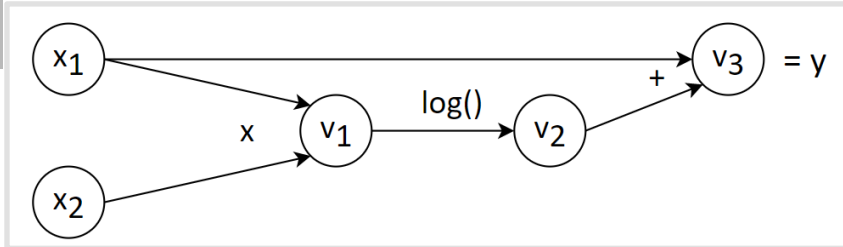


Fig1.3 :Graph of primals for function $\log(x_1 x_2) + x_1 = y$

Here the chain rule is

$$\begin{aligned} \partial_{x_1} y &= \partial_{x_1} v_3 = \partial_{x_1} (v_2 + x_1) \\ &= \partial_{x_1} v_1 \cdot \partial_{v_1} v_2 + \partial_{x_1} x_1 \end{aligned}$$

Forward approach, here $\dot{v} = \partial_{x_1} v$

Forward Primals		Forward Tangents	
x_1	5	$\dot{x}_1$	1
x_2	2	$\dot{x}_2$	0
$v_1 = x_1 x_2$	10	$\dot{v}_1 = x_2$	2
$v_2 = \log(v_1)$	1	$\dot{v}_2 = \frac{\dot{v}_1}{v_1}$	1/5
$v_3 = v_2 + x_1$	6	$\dot{v}_3 = \dot{v}_2 + 1$	1/5 + 1

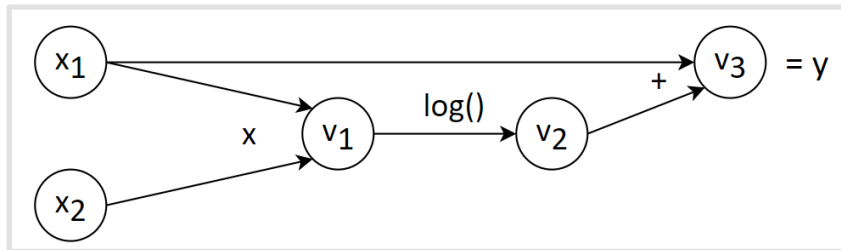
Forward for few inputs, large outputs : From the ground up.

Appendix : Example for intuition

$$f(x_1, x_2) = \log(x_1 x_2) + x_1 = y$$

$$\partial_{x_1} f(x_1, x_2) = \frac{1}{x_1} + 1$$

$$\partial_{x_1} f \Big|_{x_1=5, x_2=2} = ? = \frac{1}{5} + 1$$



Here the chain rule is

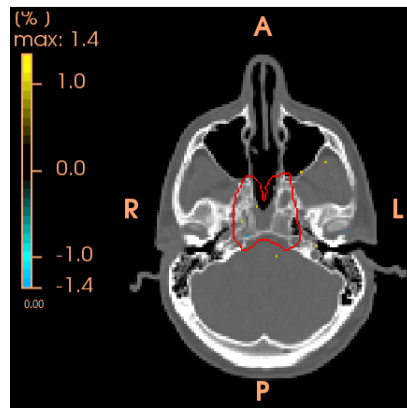
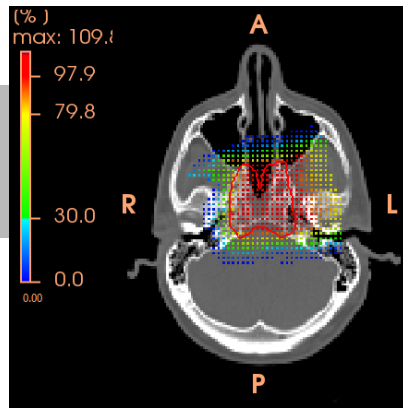
$$\begin{aligned}
 \partial_{x_1} y &= \partial_{v_1} v_3 \cdot \partial_{x_1} v_1 + \partial_{x_1} v_3 \cdot \partial_{x_1} x_1 \\
 &= \bar{v}_1 \partial_{x_1} v_1 + 1 = \bar{v}_2 \cdot \partial_{v_1} v_2 \cdot \partial_{x_1} v_1 + 1 \\
 &= \bar{v}_3 \cdot \partial_{v_2} v_3 \cdot \partial_{v_1} v_2 \cdot \partial_{x_1} v_1 + 1
 \end{aligned}$$

Backward approach, here $\bar{v} = \partial_v y$

Forward Primals		Backward Adjoint(derivatives)	
x_1	5	$\bar{x}_1 = \partial_{x_1} y = 1 + \partial_{x_1} v_1 \cdot \bar{v}_1$	$1 + 2 \cdot 1/10 = 1 + 1/5$
x_2	2	$\bar{x}_2 = \partial_{x_2} y = \bar{v}_1 \cdot \partial_{x_2} v_1$	$1/10 \cdot 5 = 1/2$
$v_1 = x_1 x_2$	10	$\bar{v}_1 = \partial_{v_1} y = \bar{v}_2 \cdot \partial_{v_1} v_2$	$1 \cdot 1/10$
$v_2 = \log(v_1)$	1	$\bar{v}_2 = \partial_{v_2} y = \partial_{v_3} y \cdot \partial_{v_2} v_3 = \bar{v}_3 \cdot \partial_{v_2} v_3$	$1 \cdot 1 = 1$
$v_3 = v_2 + x_1$	6	$\bar{v}_3 = y$	1

Backward for
large inputs,
few outputs :
From top to
bottom.

Appendix - Sum Up patients

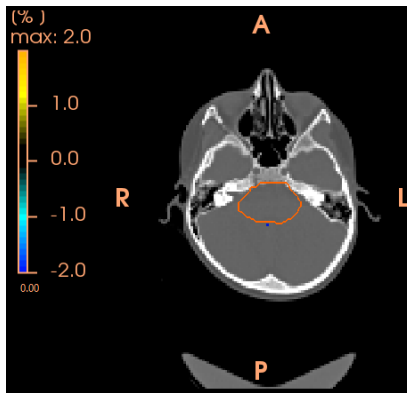
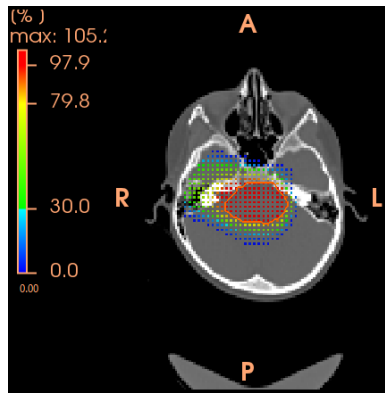


Patient 0 - Plan

Gantry angle[degree]	75.0
Couch angle[degree]	15.0
Prescribed dose [Gy]	73.8
Nozzle Extraction	15.0
Target structure	TECHPTV1

Patient 0 - Statistics

Max pos. diff. [%]	1.355
Max neg. diff. [%]	-1.355
Mean diff. [%]	0.001
Voxel within $\pm 1\%$ [%]	98.724
Voxel within $\pm 3\%$ [%]	100
Gamma Pass rate [%]*	100

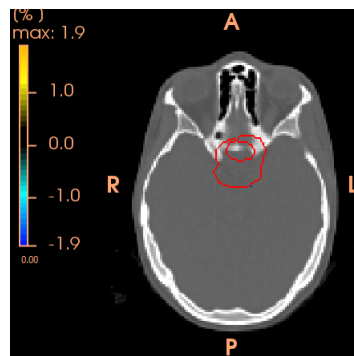
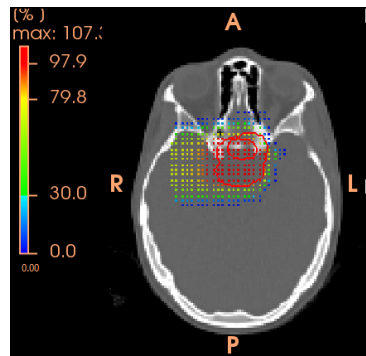


Patient 1 - Plan

Gantry angle[degree]	70.0
Couch angle[degree]	165.0
Prescribed dose [Gy]	50.4
Nozzle Extraction	1.0
Target structure	PTV

Patient 1 - Statistics

Max pos. diff. [%]	1.984
Max neg. diff. [%]	-1.984
Mean diff. [%]	0.001
Voxel within $\pm 1\%$ [%]	99.198
Voxel within $\pm 3\%$ [%]	100
Gamma Pass rate [%]*	100



Patient 2 - Plan

Gantry angle[degree]	95.0
Couch angle[degree]	180.0
Prescribed dose [Gy]	52.2
Nozzle Extraction	15.0
Target structure	TECHPTV1

Patient 2 - Statistics

Max pos. diff. [%]	1.917
Max neg. diff. [%]	-1.915
Mean diff. [%]	0.001
Voxel within $\pm 1\%$ [%]	99.472
Voxel within $\pm 3\%$ [%]	100
Gamma Pass rate [%]*	100